

Lecture Summary

Data types

Recall that a *data type* is set of values and set of operations on those values. It determines the possible values that can be stored in a variable and the operations that can be performed on that variable. It also determines the amount of memory required to store the value and the way the value is stored in the memory. Some commonly used data types in C++ are following¹:

- **int** - stores integers (whole numbers), without decimals, such as 123 or -123 . The **int** data type stores values that are within a range of $-2,147,483,648$ to $2,147,483,647$ (32-bit signed integer).

Other useful data types for integers are: **short** (stores 16-bit integers), **long** (stores 32-bit² integers), **long long** (stores 64-bit integers).

- **double** - stores floating point numbers, with decimals, such as 19.99 or -19.99 . The **double** data type is a double-precision 64-bit floating point.

Other useful data types for floating point numbers are: **float** (stores 32-bit floating point numbers), **long double** (stores 128-bit floating point numbers).

- **bool** - stores values with two states: **true** or **false**. Comparisons such as checking equality (**a == b** or **a != b**) or inequalities (**a < b** or **a >= b**) evaluate as a **bool**, so either **true** or **false**.

- **char** - stores single characters, such as **'a'** or **'B'**. The **char** values are surrounded by single quotes.

The **char** is a type of integer, so it also stores numbers, which are interpreted as characters according to the ASCII encoding. For example, the ASCII value for the character **'a'** is 97 and the ASCII value for the character **'b'** is 98.

- **std::string** - stores text, such as **"Hello World"**. The **std::string** values are surrounded by double quotes.

Casting variables

Casting converts a value from one data type to another. A cast can be *implicit*, which means that C++ compiler performs the conversion automatically, or *explicit*, which means that the programmer requests the conversion. Implicit conversions are common when a value is assigned to a variable of a different type:

```
int temperature = 21;
```

```
double preciseTemperature = temperature; // implicit int to double
```

The value of **preciseTemperature** is **21.0**, allowing for representing fractional parts in the temperature which was not possible with **temperature**.

An explicit conversion using the old C-style cast places the target type in parentheses before the value (you can also place the value in the parentheses and target type before it):

¹The sizes of these data types may vary from one compiler to another.

²32-bit on Windows, 64-bits on Linux/MacOS

```
int completedTasks = 7;
int totalTasks = 10;
double completionRate = (double) completedTasks / totalTasks;
// the RHS is the same as doing double (completedTasks) / totalTasks
// completionRate is 0.7, rather than 0 (integer division)
```

Because one operand is converted to **double** before the division, the result keeps its fractional part. Explicit casts can also intentionally discard a fractional part, such as when converting a measurement to an integer:

```
double money = 7.0;
double cost = 2.0;
int can_buy = money / cost;
// because the type of can_buy is int, the fractional part of 7.0/2.0 = 3.5 is
// discarded and the value stored is 3
```

Excerpts from the C++ library

function	description	example	library
min()	returns the minimum of two numbers	int m = std::min(3, 5);	<algorithm>
max()	returns the maximum of two numbers	std::cout << std::max(3, 5);	<algorithm>
abs()	returns the absolute value of a number	int x = std::abs(-10);	<cmath>
pow()	returns the first argument raised to the power of the second argument	double y = std::pow(2, 3)	<cmath>
sqrt()	returns the square root of a number	double s = std::sqrt(16);	<cmath>
ceil()	returns the smallest integer that is greater than or equal to the argument	int z = std::ceil(9.2);	<cmath>
floor()	returns the largest integer that is less than or equal to the argument	int z = std::floor(9.2);	<cmath>
lround()	returns the long integer that is closest to the argument	long z = std::lround(9.2);	<cmath>
to_string()	convert to std::string value	std::string x = std::to_string(17);	<string>
stoi()	convert std::string value to int value	int x = std::stoi("17");	<string>

You need to add appropriate library file to be able to use a function. For instance:

```
#include <iostream>
#include <cmath>
int main() {
    std::cout << "Square root of 2 is: "
               << std::sqrt(2);
}
```

WSL for C++ programming

Ubuntu (a Linux-based OS) is installed on all lab PCs via WSL 2.0. Think of it as a virtual operating system installed in some portion of memory.

1. **Open Ubuntu.** From the Windows Start menu, search for and open **Ubuntu**.
2. **Understand the prompt.** Ubuntu displays a prompt in the following form:

```
<username>@<hostname>:<pwd>$
```

The username and hostname are not important for you as they have been pre-configured. The **pwd** part is important: it shows your *current working directory*. By default when you open Ubuntu, this going to be `~`. Always check that you are working inside the intended working directory (that is, your created lab folder) before creating, editing, compiling, or running a file.

3. **Create and switch to your lab folder.** Make a fresh folder for each lab using the name **Lab_<Number>_<ERP>**. Replace **<Number>** with the lab number and **<ERP>** with your ERP number. For example, for ERP 27081 and lab 3:

```
mkdir ~/Lab_3_27081
cd ~/Lab_3_27081
```

The **mkdir** command makes a new directory and the argument **~/Lab_3_27081** makes sure that the lab directory is created at the correct level (in `~`, which is shorthand for `/home/username/`). The **cd** command changes into the folder specified by its argument. Only work in this folder. If you close Ubuntu and reopen it, it will launch back in with the current working directory set to `~`, use **cd** to move to your lab folder and check the prompt (from point 2) to make sure you are in the correct folder.

4. **Create a source file.** Use **touch** to create an empty C++ file (make sure you are in the correct folder!):

```
touch q1.cpp
```
5. **Open and save the file.** Open the file in Notepad with:

```
notepad.exe q1.cpp
```

The terminal will appear to hang until you close Notepad. Type your program in Notepad and press **Ctrl+S** to save it. Do not use Notepad's **Save As** option: if you do so, then you would be required to save the file in the correct working directory as accessed by your Ubuntu shell and not in some common place like Desktop or Documents. Using **Ctrl+S** ensures the file is saved in the correct place. Close Notepad when you finish editing.

6. **Rename a file if necessary.** If you made a mistake in the filename, rename it with `mv`:

```
mv wrong_name correct_name
```

7. **List down files in your folder.** If you want to list down all the files in your folder, use `ls`:

```
ls
```

8. **Compile and run the program.** Compile the source file with `g++`, then run the resulting program:

```
g++ source_file -o output_file
./output_file
```

If compilation reports errors, reopen the source file, correct it, save with `Ctrl+S`, and compile again. Keep the terminal in your lab folder throughout this process.

Submission Instructions

Follow these instructions carefully; marks may be deducted otherwise.

- Source code files for each attempted question must have the file name `q<question_number>.cpp` where `<question_number>` is to be replaced by the index of the question as given in this lab manual. For example, the files for questions 1, 2, and 3 will be stored as `q1.cpp`, `q2.cpp`, and `q3.cpp` respectively.
- The lab must be completed by 30 minutes before the scheduled end time for the lab. A viva will be conducted for all lab questions for grading. You are required to have your source files compiled before the viva and be ready to run the compiled binaries.
- After your viva, you are still required to submit your code to LMS in the form of zip archive. Zip **ONLY source files** (*.cpp) in an archive named `Lab_<lab_number>_<ERP>.zip`. For example, with ERP 27081 and lab 3, the zip would be called `Lab_3_27081.zip`. To zip your lab folder from the terminal, use `zip archive_name q1.cpp q2.cpp q3.cpp q4.cpp q5.cpp` to make an archive with the name `archive_name`.

Lab Questions

Ensure that you take input of the correct data type for all questions and that your output matches the expected output format. Marks will be deducted for not following these instructions.

1. **What goes where?** Your friend hands you three numbers and asks you to arrange them in order of smallest to largest.

Create a program that takes in three numbers a , b , and c of type `int` as input, each on a new line, via `cin` and prints them out in ascending order separated by a space to `cout`. Your program cannot use an `if` or `switch` statement.

Example: Below are examples of the form of the expected output based on the given input.

Sample Input	Expected Output
2 -4 6	-4 2 6
43 22 5	5 22 43

2. **Even or Odd:** Your friend has challenged you to a version of scrabble in which each of you only has access to half of the alphabet. More specifically, you have access to all of the odd-indexed alphabets such as a (index 1), c (index 3), e (index 5), and so on, and your friend has access to all of the even-indexed alphabets such as b (index 2), d (index 4), f (index 6), and so on. Before you can play this game however, you want to divide the scrabble tiles into even and odd alphabets.

Create a program that takes as input a letter c of type `char` and outputs -1 if the letter is an odd-indexed character and 1 if the letter is an even-indexed character to `cout`. You may assume c is always a lower-case character and nothing else. Your program cannot use an `if` or `switch` statement.

Hint 1: If you cast a `char` variable to `int`, you can get the ASCII value of that character which serves as its numeric representation. You may try outputting these values to `cout` for some letters to see their range before writing your actual logic.

Hint 2: If you cast a `bool` variable to `int`, `true` gets converted to 1 and `false` gets converted to 0. This can be used cleverly along with simple arithmetic to get the behavior of `if` statements.

Example: Below are examples of the form of the expected output based on the given input.

Sample Input	Expected Output
a	-1
h	1

3. **In-between:** You recently won not one but two lotteries. Suddenly, a friend that hasn't spoken to you in a decade texts you wishing you and your family good health and asks to hangout. When you show up, you realize that he wants exactly half of the money you have won across both lotteries. He claims that he once lent you a pencil when you were in primary school, so now you must pay him back (adjusting

for inflation, of course). You compute both of yours share, but he still believes you are trying to trick him.

Create a program that takes in three positive numbers $winnings_A$, $winnings_B$, and $share$ of type `unsigned int` as input and outputs `1` if the $share$ is correct, i.e., it is the arithmetic mean of $winnings_A$ and $winnings_B$, else it outputs `0` to `cout`. Assume that $winnings_A < winnings_B$ and that $winnings_A + winnings_B$ is an even number. **You MUST use only the `unsigned int` data type to store the input AND any intermediate calculations. The program also cannot use any `if` or `switch` statements**

Hint: Think of an equivalent expression for the arithmetic mean that does not cause the same issues the standard expression $(\frac{a+b}{2})$ will cause.

Example: Below are examples of the form of the expected output based on the given input.

Sample Input	Expected Output
4 8 5	0
4294967293 4294967295 2147483646	1